

How the PDB + AlphaFold Script Works

Simple step-by-step explanation of Broad_PDB_Search(1).ipynb

This guide explains what happens after you type a protein name, especially how the script understands that name, finds the correct protein identity, searches for PDB structures even when the PDB uses different names, removes unrelated structures, collects structure-quality information, and finally creates the Excel workbook.

Main example used: CDK4

The notebook output shown in the uploaded file resolves **CDK4** to UniProt accession **P11802**, with the official protein name **Cyclin-dependent kinase 4**. This is a useful example because the user's short input and the names used in databases are not necessarily identical.

1. The whole workflow in one picture

Step	What the script does
1	You enter a protein name, gene symbol, alias, or UniProt accession.
2	The script searches UniProt to identify exactly which human protein you mean.
3	UniProt gives the stable identity: accession, official name, gene symbols, aliases and sequence.
4	The script sends several types of searches to RCSB PDB: UniProt accession, gene symbols, and meaningful protein/entity names.
5	All returned PDB IDs are combined into one unique candidate list.
6	Each candidate is checked: X-ray? human? protein? correct UniProt/gene/name? sequence consistent?
7	Only qualifying target structures continue.
8	For each qualifying PDB, the script collects resolution, chains, residues, R-values, validation metrics, ligands and activity.
9	Independently, it always searches AlphaFold using the resolved UniProt accession.
10	Everything is written into Excel, followed by quality scoring/ranking.

2. What happens when you type CDK4?

The script starts with:

```
protein_name = input("Enter protein name: ").strip()
```

So if you type **CDK4**, the value stored in the variable **protein_name** is simply the text **CDK4**. The script does not yet know whether CDK4 is a gene, protein name, alias, or accession. It must resolve that identity next.

It also checks that the input is not empty. If you press Enter without typing anything, the script stops with an error.

Important: the input is not directly used as the final PDB search key. The script first converts the user's input into a stronger biological identity.

3. The most important part: how the script recognizes the protein

The script uses **UniProt** as the identity-resolution step. Think of UniProt as the identity card for the protein.

For CDK4, the script performs several UniProt searches. The important idea is that it does not depend on only one spelling.

Possible input type	Example	What the script searches
Gene symbol	CDK4	gene_exact:CDK4 and human
Gene search	CDK4	gene:CDK4 and human

Protein name	Cyclin-dependent kinase 4	protein_name:"Cyclin-dependent kinase 4" and human
UniProt accession	P11802	accession:P11802 and human

The script also recognizes an input that looks like a UniProt accession using a pattern check. If it looks like an accession, it searches that accession first.

Then it collects the possible names from each UniProt result: recommended protein name, short names, alternative names, UniProt ID, gene names and gene synonyms.

4. Why does it collect aliases and different names?

This is the key to understanding how the script can find PDB structures whose names are different from your input.

Example:

You type: **CDK4**
 UniProt official name: **Cyclin-dependent kinase 4**
 Gene symbol: **CDK4**
 UniProt accession: **P11802**

A PDB entry may describe a protein as something like **Cyclin-dependent kinase 4**, **CDK4**, a kinase-domain construct, or a related construct description. Therefore, searching only the exact text **CDK4** can miss useful entries.

The script builds a set called **TARGET_NAMES**. It puts the user's input, official name, alternative names, short names and other collected names into that set. It also builds **TARGET_GENES** for gene symbols and synonyms.

So the script changes the problem from: *"Find PDB entries containing exactly CDK4"* to: *"Find structures connected to the biological identity represented by UniProt P11802 and its known names."*

5. How the script searches the PDB

After UniProt resolution, the script has a strong identity. It then searches the RCSB PDB in multiple routes. This is important because different PDB records may contain different naming information.

Route 1 — UniProt accession

The script searches the RCSB PDB for the exact UniProt accession. For CDK4 this is **P11802**. This is the strongest search route because an accession is much more specific than a common protein name.

Route 2 — Gene symbols

The script searches the RCSB field for the gene name. For CDK4, it searches **CDK4** and any collected gene synonyms.

Route 3 — Polymer entity names

The script also searches meaningful protein/entity descriptions. It deliberately avoids very short names in this free-text search because short strings can produce many unrelated matches.

The three routes are combined using a Python **set**. A set automatically removes duplicates. Therefore, if the same PDB ID is found by UniProt and gene search, it appears only once in the candidate list.

6. What is a PDB entry versus a PDB entity?

This distinction is very important.

PDB entry: the complete experimental structure record, identified by a PDB ID such as 1ABC.

Polymer entity: one unique biological molecule/protein sequence inside that entry. One PDB entry can contain several entities, such as the target protein, another protein, DNA, etc.

Chain/asym ID: a physical chain instance of an entity in the structure.

The script first finds PDB entry IDs, then opens the entry and examines its polymer entities. This is how it can decide whether the actual protein in the structure is your target.

7. Why candidate discovery alone is not enough

Suppose the search finds a PDB because its description contains the text “CDK4”. That does not automatically mean the structure contains CDK4. The name could refer to an inhibitor, another interacting molecule, or a misleading description.

Therefore, the script has a separate **strict target-entity validation** stage.

Check	Simple meaning
Protein type	Is the entity actually a protein, not DNA/RNA or another polymer?
Human source	Is it from Homo sapiens / taxonomy 9606?
UniProt accession	Does the entity reference P11802?
Gene/name	Do the PDB entity's gene or protein names match the target identity?
Sequence	If the name match is less direct, does its sequence support the target identity?
Experimental method	Is the structure X-ray diffraction?

8. The sequence check: the safety net

The script contains a local pairwise sequence-alignment check. If a PDB entity is not an exact name/accession match but its name contains a target alias, the script does not immediately trust the name.

Instead, it compares the PDB entity sequence with the UniProt target sequence. The alignment uses local alignment. The script accepts this special name-plus-sequence route when sequence identity is at least **85%** and at least **30 aligned**

residues.

In simple language: **the name can suggest that it is the target, but the sequence must provide additional evidence.**

9. Example: CDK4 from input to PDB candidate

Stage	What happens
User input	CDK4
UniProt identity	P11802
Official name	Cyclin-dependent kinase 4
Gene symbols	CDK4 and any UniProt-listed synonyms
Target names	CDK4 + official/alternative names collected from UniProt
PDB searches	P11802 search + gene searches + meaningful entity-name searches
Candidate list	All returned PDB IDs are merged and duplicates removed
Candidate validation	Each PDB is checked for human X-ray target entity
Final PDB list	Only entries that pass the target checks continue to data extraction

Notice that the script does **not** say “CDK4 = PDB ID X” directly. Instead it builds evidence in stages: user text → UniProt identity → multiple PDB searches → target-entity validation → final structure list.

10. After a PDB passes: what data are collected?

- Experimental method — the script keeps X-ray diffraction entries.
- Resolution — read from the RCSB entry information.
- Deposited and modeled residue counts — used to see how much of the deposited protein is modeled.
- PDB chains — collected from polymer entities/instances.
- Depositor R-free and R-work — read from the mmCIF file.
- R-value difference — calculated from R-free and R-work.
- wwPDB validation metrics — R-free, clashscore, Ramachandran outliers, sidechain outliers and RSRZ outliers.
- Average B-factor — extracted from the official validation PDF.
- Ligands — non-polymer components are checked and common solvent/ion components are filtered out.
- Ligand activity — binding-affinity information is collected when available.

11. Why does the script use several PDB data sources?

The script does not expect every piece of information to exist in one API response.

RCSB entry API: general entry information such as method, resolution and identifiers.

RCSB polymer entity API: protein entity details, chains, sequence and references.

mmCIF file: depositor R-work and R-free.

wwPDB validation XML: official validation metrics.

wwPDB validation PDF: Average B-factor extraction.

RCSB non-polymer entity API: ligand/component information.

12. How AlphaFold is handled

An important feature of this notebook is that AlphaFold collection is **independent of the PDB result**. In other words, AlphaFold is searched even if crystal structures are available.

The script uses the resolved UniProt accession, not the user's raw name. For CDK4, it therefore uses **P11802**.

First it tries the AlphaFold primary API. If that does not return usable data, it uses the EMBL-EBI 3D-Beacons service restricted to AlphaFold DB.

The AlphaFold output can include model identifier, chain, UniProt sequence positions and oligomeric state. The Excel sheet is called **AlphaFold_Data**.

13. Why the output names may look different

There are several different names/identifiers in the workflow, and they are not mistakes. Each has a different job.

Name / ID	Example	Purpose
User input	CDK4	What you typed
Official UniProt protein name	Cyclin-dependent kinase 4	Standard protein identity
UniProt accession	P11802	Stable database identifier
Gene symbol	CDK4	Gene-level identifier
PDB ID	e.g., 1ABC	Identifier for a particular experimental structure
PDB entity name	May describe the full protein/domain/construct	How the protein is described in that structure
PDB chain	A, B, etc.	Specific chain instance
AlphaFold identifier	AF-P11802-F1 or API-returned model ID	Identifier for the predicted model

14. How the Excel workbook is created

The workbook contains at least three important sheets:

- **PDB_Data** — one row per qualifying PDB structure.
- **AlphaFold_Data** — AlphaFold models for the resolved UniProt target.
- **Run_Info** — input, resolved UniProt, number of candidates, number of qualifying PDB structures and AlphaFold models.

The PDB ID is converted into a clickable RCSB link. AlphaFold identifiers are also linked to the AlphaFold entry page.

15. The final scoring section

After the data collection workbook is created, the notebook contains a separate PDB quality-scoring section. It creates multiple scoring hypotheses with different weights.

The three example hypotheses in the script are:

Hypothesis	Resolution	R-difference	Residue	Validation	B-factor
H1	25%	25%	10%	20%	20%
H2	20%	20%	10%	25%	25%
H3	20%	20%	20%	20%	20%

Lower values are treated as better for resolution, residue-count difference, R-value difference and average B-factor. The script also calculates a validation-quality status using the median of the validation metrics.

This produces ranked tables such as **PDB_Scoring_H1**, **PDB_Scoring_H2** and **PDB_Scoring_H3**, plus a **Ranges** sheet.

16. The most important concept: candidate search vs final selection

Candidate search asks: “Could this PDB be related to my protein?” It intentionally uses multiple routes to avoid missing structures.

Target validation asks: “Does this PDB actually contain the correct human target protein?” This removes false positives.

Data extraction asks: “What are the experimental and quality properties of this valid structure?”

Scoring asks: “Among the valid structures, which ones look better for the intended structural study?”

This separation is good design. The script should not select a structure merely because its name looks similar.

17. A very simple analogy

Imagine you ask a librarian: “**Find all books about CDK4.**”

- **Input:** CDK4 — your request.
- **Identity check:** the librarian confirms the exact subject identity using UniProt.
- **Aliases:** the librarian also learns that the subject may be written as Cyclin-dependent kinase 4.
- **Search:** the librarian searches by accession, gene name and protein names.
- **Candidate list:** all matching books/records are collected.
- **Verification:** each record is checked to make sure it is actually about the target.
- **Quality:** useful details are extracted.
- **Ranking:** the best records are ranked.

That is essentially what the notebook is doing, but with biological databases instead of books.

18. What the script is NOT doing

- It is not taking the user input and directly guessing a PDB ID.
- It is not searching only the exact text typed by the user.
- It is not accepting every PDB returned by a name search.
- It is not stopping AlphaFold search just because crystal structures exist.
- It is not treating every molecule in a PDB as the target protein.
- It is not using only one database field to decide whether a PDB is correct.

19. One-line mental model to remember

User protein name → UniProt identity → names/aliases + accession + sequence → multiple PDB searches → unique candidates → target/entity validation → PDB metadata + validation + ligands → independent AlphaFold search → Excel → quality scoring/ranking.

20. Final takeaway for explaining this script to your manager

You can explain the protein-recognition part like this:

“The user can enter a protein name, gene symbol, known alias or UniProt accession. The script first resolves that input through UniProt and obtains a stable UniProt accession, official protein name, gene names, aliases and protein sequence. It then uses the accession, gene symbols and meaningful protein names to search RCSB PDB through multiple routes. All candidate PDB IDs are merged and deduplicated. Each candidate is then

validated at the polymer-entity level to confirm that it is a human protein and corresponds to the target, using UniProt references, names/genes and, when necessary, sequence similarity. Only validated X-ray structures are processed further. AlphaFold is searched separately using the resolved UniProt accession, so predicted models are collected even when crystal structures are available.”

This is the central logic of the notebook. The many different names you see are different identifiers for the same biological target or for specific structure records; the UniProt accession acts as the main stable bridge between them.